MAKERERE UNIVERSITY
COLLEGE OF COMPUTING AND INFORMATION SCIENCES

Object Oriented Programming with Java

Inventory Management System

Project Documentation

Group Name:	[e.g., Group 5 - kafeero_marvin_day]
Group Leader:	[Kafeero Marvin - Reg No]
Members:	1. [Name - Reg No] 2. [Name - Reg No] 3. [Name - Reg No]
Course:	Object Oriented Programming (Java)
Lecturer:	[Lecturer Name]
GitHub Link:	https://github.com/[username]/InventoryManagementSystem
Date:	November 2026

Submission: MUELE - Deadline 12th Nov 2026

Page Break

1. Description of the Application

The Inventory Management System is a simple Java Swing desktop application designed to help small businesses manage stock. It allows users to add new products, view all products in a table, update quantities, delete products, search by name/category/ID, and monitor low-stock items and total inventory value. Data is stored in arrays (Product[] and Supplier[]) in memory as per assignment requirements, no database needed. The application demonstrates OOP concepts required in the course.

2. Main Features

• Add Product with validation (duplicate ID check, negative values check)
• View Products in JTable with status (In Stock / LOW STOCK)
• Update Quantity by Product ID
• Delete Product with confirmation
• Search Products by keyword (name, category, ID)
• Show All Products, Clear Form
• Automatic statistics: Total Products, Total Inventory Value (UGX), Low Stock Count
• Click on table row to auto-fill form for editing
• Sample data pre-loaded

3. Classes and Their Purpose

Class	Purpose	Key Attributes/Methods
Product	Represents a meaningful entity - an item in inventory. Demonstrates encapsulation, getters/setters, constructors, access modifi	id, name, category, quantity, price, supplierId getTotalValue(), isLowStock(), toString()
Supplier	Second entity - supplier of products. Shows relationship between entities (Product has supplierId).	supplierId, supplierName, contact Constructor, getters/setters
Inventory	Manager class that stores and manages arrays of Product and Supplier objects. Implements business logic using loops and conditionals.	Product[] products (100), Supplier[] suppliers addProduct(), deleteProduct(), updateQuantity(), searchProducts(), getLowSto
MainFrame	GUI class using Java Swing. Provides user interaction, displays data in table, handles events.	JFrame, JTable, JTextField, JComboBox ActionListeners, refreshTable(), addProductAction() etc.
Main	Entry point. Contains main() method and group details as comments.	main() launches SwingUtilities.invokeLater. Sets Look and Feel.

4. How Classes Interact

- Main creates MainFrame. - MainFrame creates one Inventory object. - Inventory holds Product[] and Supplier[] arrays. Composition relationship. - When user clicks Add, MainFrame creates a new Product object (new Product(...)) and passes it to inventory.addProduct(Product). - Inventory uses for loops to check duplicates, and if/else to validate. - searchProducts uses for loop + if to filter arrays. - getTotalInventoryValue uses while loop to sum values. - getLowStockProducts uses for loops with conditional quantity < 10. - Product.isLowStock() is called by Inventory and MainFrame to decide status. - Table displays Product data retrieved via getters (getId(), getName() etc.). - Supplier objects are used to populate Supplier combo box. This satisfies: At least 3 classes, appropriate attributes/methods, constructors, access modifiers (private fields), getters/setters, object creation and use.

5. Flow Control and Arrays Used

Arrays: Product[] products = new Product[100]; Supplier[] suppliers; Managed with productCount.
Conditional: if (fields empty) show error; if (duplicate) return false; if (low stock) show RED status; switch(category) for custom message.
Iterative: for loop for searching, deleting (with shift), adding; while loop for total value; do-while could be demonstrated in console version but for loop is primary here.
GUI: Java Swing - JFrame, JPanel, JButton, JTextField, JComboBox, JTable with DefaultTableModel, JOptionPane.

6. Screenshots (Add your own after running)

Figure 1: Main GUI with form, table, and stats
Figure 2: Add Product Success
Figure 3: Low Stock highlighting

(Run the program and take screenshots with Snipping Tool and paste here - not required but adds marks)

7. How to Run

javac *.java
java Main
Requires JDK 8 or above. No external libraries.